BÁO CÁO PHẦN BACKEND

Hệ thống gợi ý từ khóa bằng khoảng cách Levenshtein

Môn học	Phân tích và Thiết kế Giải thuật — Bài tập giữa kỳ
Phần thực hiện	Backend (REST API)
Dữ liệu	`dictionary.txt` — 23.850 từ khóa sản phẩm tiếng Việt
Công nghệ	Python 3.10+, FastAPI, Uvicorn, Pytest
Ngày hoàn thành	02/11/2026

Mục lục

1. Tóm tắt
2. Mục tiêu và phạm vi
3. Phân tích yêu cầu
4. Thiết kế hệ thống
5. Thuật toán và các tối ưu
6. Kiểm thử
7. Kết quả đo hiệu năng
8. Hạn chế và hướng phát triển
9. Kết luận
10. Phụ lục

---

1. Tóm tắt

Phần backend biến thuật toán gợi ý từ khóa của nhóm (`algorithm.py`, `data_processor.py`) thành một dịch vụ HTTP chạy được thật: nạp từ điển 23.850 từ khóa một lần khi khởi động, nhận truy vấn có thể sai chính tả và trả về top-5 từ khóa gần nhất theo khoảng cách Levenshtein.

Điểm khó không nằm ở công thức quy hoạch động — công thức đã có sẵn — mà ở chỗ: quét toàn bộ từ điển cho mỗi truy vấn tốn khoảng 600 ms, quá chậm cho một API. Backend bổ sung ba lớp tối ưu **không làm thay đổi kết quả** (lọc ứng viên bằng trigram, quy hoạch động có chặn, quét dự phòng theo độ dài) cùng bộ đệm LRU, đưa thời gian phản hồi điển hình xuống 2,5–25 ms.

Chỉ số	Kết quả
Kích thước từ điển	23.850 từ khóa
Thời gian khởi động (nạp + dựng chỉ mục)	~103 ms
Thời gian phản hồi điển hình	2,5–25 ms
p95 trên 20 truy vấn đo	49,2 ms
Nhanh hơn bản quét toàn bộ	~26,7 lần (trung bình)
Cache hit	0,016 ms
Kiểm thử	117 test, đạt 100%, độ phủ 100%
Đối chiếu với bản gốc	top-5 khớp 100%

---

2. Mục tiêu và phạm vi

2.1 Mục tiêu

1. Đóng gói thuật toán có sẵn thành REST API dễ gọi từ frontend.
2. Giữ **đúng tuyệt đối** kết quả của bản thuật toán nhóm đã viết.

3. Giảm thời gian phản hồi từ hàng trăm mili giây xuống mức dùng được trong thực tế.
4. Có kiểm thử tự động, tài liệu và số đo hiệu năng để báo cáo.
5. Chạy được trên máy sinh viên chỉ với Python, không cần dịch vụ ngoài.

2.2 Phạm vi

- Trong phạm vi: API, xử lý nghiệp vụ, chuẩn hóa tiếng Việt, kiểm thử, đo hiệu năng, trang demo nội bộ để thử tay.
- Ngoài phạm vi: giao diện người dùng chính thức (phần của thành viên khác), cơ sở dữ liệu, xác thực người dùng.

---

3. Phân tích yêu cầu

3.1 Yêu cầu chức năng

Mã	Yêu cầu	Cách đáp ứng
F1	Nhận từ khóa người dùng nhập, kể cả sai chính tả	`POST /api/v1/suggest`, chuẩn hóa trước khi so khớp
F2	Trả về tối đa 5 từ khóa gần nhất	Tham số `limit` (1–20), mặc định 5
F3	Kết quả sắp xếp theo khoảng cách, ổn định khi bằng điểm	Sắp theo `(distance, keyword)` tăng dần
F4	Giới hạn khoảng cách tối đa (tùy chọn)	Tham số `max_distance`
F5	Kiểm tra dịch vụ còn sống	`GET /api/v1/health`
F6	Số liệu vận hành phục vụ báo cáo	`GET /api/v1/stats` (kích thước từ điển, cache, thời gian nạp)

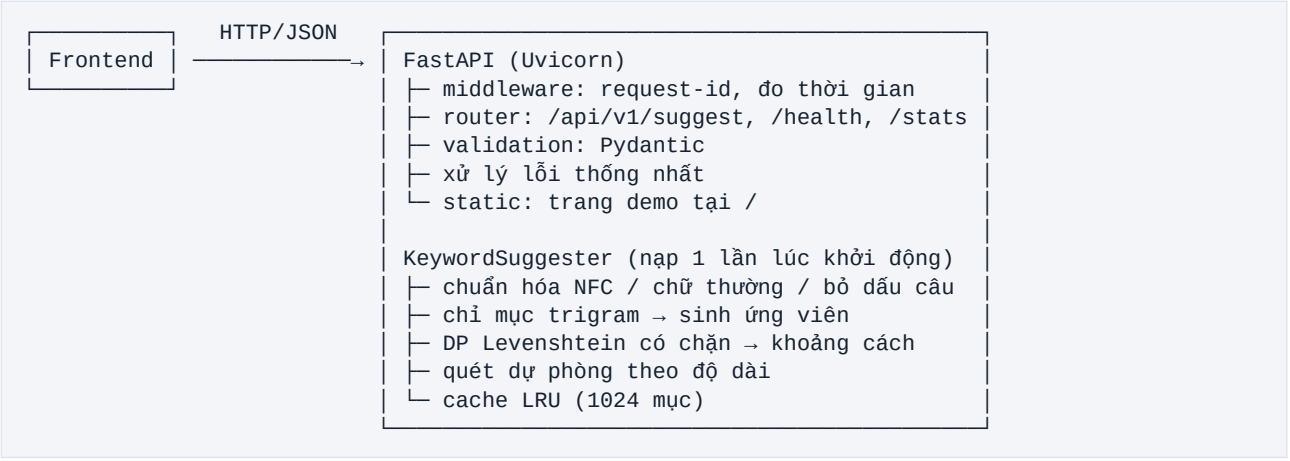
3.2 Yêu cầu phi chức năng

Mã	Yêu cầu	Kết quả
N1	Thời gian phản hồi < 100 ms với truy vấn thông thường	p95 = 49 ms
N2	Kết quả không sai lệch so với thuật toán gốc	100% khớp, có test đối chiếu
N3	Chịu được nhiều request đồng thời	30 request song song, 0 lỗi
N4	Dễ cài đặt, chạy trên Windows/Linux	Chỉ cần Python + pip
N5	Dễ bảo trì, dễ đọc	Tách lớp rõ ràng, 100% độ phủ test

---

4. Thiết kế hệ thống

4.1 Kiến trúc tổng thể



Toàn bộ từ điển được nạp và chuẩn hóa **một lần** trong sự kiện `lifespan` của FastAPI, sau đó chỉ đọc. Mỗi request không phải đọc lại tệp hay chuẩn hóa lại 23.850 từ khóa — đây là khác biệt lớn so với bản demo cũ.

4.2 Luồng xử lý một truy vấn

1. **Nhận request** → middleware gắn `X-Request-ID`, bấm giờ.
2. **Kiểm tra dữ liệu** bằng Pydantic: độ dài 1–200, `limit` 1–20, `max_distance` 0–100; truy vấn rỗng hoặc chỉ có ký tự đặc biệt bị từ chối với lỗi 422.
3. **Chuẩn hóa** truy vấn: NFC → chữ thường → bỏ dấu câu → gộp khoảng trắng.
4. **Tra cache**: nếu trùng `(truy vấn chuẩn hóa, limit, max_distance)` thì trả ngay.
5. **Tìm gợi ý**: nới ngưỡng `d = 1, 2, 3, ...`; mỗi bước lấy ứng viên từ chỉ mục trigram, tính khoảng cách có chặn, giữ top-k. Đủ `k` kết quả thì dừng.
6. **Đóng gói** response kèm `took_ms`, `cached`, `evaluated_candidates`; ghi log.

4.3 Thiết kế API

Phương thức	Đường dẫn	Chức năng
POST	`/api/v1/suggest`	Gợi ý từ khóa, body JSON
GET	`/api/v1/suggest?q=...`	Gợi ý từ khóa, tiện thử bằng trình duyệt
GET	`/api/v1/health`	Kiểm tra dịch vụ
GET	`/api/v1/stats`	Thống kê từ điển, cache, thuật toán
GET	`/`, `/docs`	Trang demo và Swagger UI

Ngoài nhóm `/api/v1`, backend còn phục vụ nhóm đường dẫn gọn theo hợp đồng mà frontend liệt kê trong `UI/src/API_List.md` (xem mục 4.5). Hai nhóm gọi lại cùng logic nên kết quả không bao giờ lệch nhau.

Ví dụ request:

```
{
  "query": "điện thoại samsng",
  "limit": 5
}
```

Ví dụ response:

```
{
  "query": "điện thoại samsng",
  "normalized_query": "điện thoại samsng",
  "suggestions": [
    { "keyword": "điện thoại samsung", "distance": 1 },
    { "keyword": "điện thoại gaming", "distance": 2 },
    { "keyword": "điện thoại asus", "distance": 4 },
    { "keyword": "điện thoại mini", "distance": 4 },
    { "keyword": "điện thoại msi", "distance": 4 }
  ],
  "count": 5,
  "limit": 5,
  "max_distance": null,
  "took_ms": 11.7,
  "cached": false,
  "evaluated_candidates": 186
}
```

Hai trường `took_ms` và `evaluated_candidates` không bắt buộc với frontend nhưng rất hữu ích khi báo cáo: chúng chứng minh bộ lọc chỉ phải tính khoảng cách cho 186 trong 23.850 từ khóa.

4.5 API bổ sung theo hợp đồng của frontend

Sau khi nhóm hoàn thành khung giao diện (commit `2bfd34b`, thư mục `UI/`), backend được mở rộng thêm 10 đường dẫn mà `UI/src/API_List.md` yêu cầu. Nhóm này không có tiền tố `/api/v1` và gọi lại đúng logic cũ, nhờ vậy hai hợp đồng luôn nhất quán.

Phương thức	Đường dẫn	Chức năng	Tương đương	
GET	`/health`	Kiểm tra dịch vụ	`/api/v1/health`	
GET	`/search/suggest?q=...`	Gợi ý mờ khi gõ sai	`/api/v1/suggest?q=...`	
GET	`/search?q=...&limit=20&mode=contains\`	prefix`	Tìm từ khóa chứa/bắt đầu bằng chuỗi	mới
GET	`/dictionary?page=1&page_size=50&q=&length=&sort=`	Duyệt từ điển, phân trang, lọc	mới	
GET	`/dictionary/stats`	Thống kê từ điển, thuật toán, cache	`/api/v1/stats`	
GET	`/algorithm/distance?source=...&target=...`	Khoảng cách Levenshtein giữa hai chuỗi	mới	
POST	`/algorithm/visualize`	Bảng quy hoạch động đầy đủ + đường đi tối ưu	mới	
POST	`/benchmark/runs`	Chạy benchmark, lưu theo `run_id`	mới	
GET	`/benchmark/runs/{run_id}`	Xem lại kết quả một lần chạy	mới	
GET	`/benchmark/runs/{run_id}/export`	Tải CSV (UTF-8 BOM, mở bằng Excel)	mới	

Vài quyết định thiết kế đáng chú ý:

1. **`/search` tách khỏi `/search/suggest`.** Ô nhập liệu dùng gợi ý mờ (tính khoảng

cách), còn trang danh sách chỉ cần khớp chuỗi con — nhanh hơn nhiều và trả về `total` đầy đủ. Kết quả sắp theo vị trí khớp, rồi độ dài, rồi bảng chữ cái.

2. **`/algorithm/visualize` trả cả bảng `D[i][j]` và đường đi tối ưu** (match / substitute / insert / delete). Bảng giới hạn 61×61 ô để một request không thể làm nghẽn máy chủ.

3. **`/benchmark/runs` đo nguội**: bộ đệm LRU bị xóa trước mỗi lần đo, nên một lần trúng cache (0,02 ms) không bị ghi nhận thành hiệu năng thật của thuật toán. Tùy chọn `include_baseline` đo thêm mốc quét toàn bộ từ điển và **tự đối chiếu top-k hai bên**, trả về trường `matches_baseline`.

4. **Kết quả benchmark chỉ giữ trong bộ nhớ** (20 lần chạy gần nhất) — đủ cho demo và báo cáo, không cần cơ sở dữ liệu.

Ví dụ minh họa bảng quy hoạch động với `kitten → sitting`: 7×8 ô, khoảng cách 3, đường đi gồm 2 phép thay thế, 4 phép giữ nguyên và 1 phép chèn.

4.6 Cấu trúc mã nguồn

Tệp	Trách nhiệm
`app/main.py`	Tạo ứng dụng, lifespan, middleware, xử lý lỗi, gắn router và trang demo
`app/config.py`	Đọc cấu hình từ biến môi trường
`app/schemas.py`	Mô hình request/response và kiểm tra dữ liệu
`app/api/common.py`	Hàm dùng chung: đóng gói response gợi ý
`app/api/suggest.py`	Endpoint gợi ý (POST/GET)
`app/api/system.py`	Endpoint health và stats (+ `/health` cho UI)
`app/api/search.py`	`/search/suggest` và `/search`
`app/api/dictionary.py`	`/dictionary`, `/dictionary/stats`
`app/api/algorithm.py`	`/algorithm/distance`, `/algorithm/visualize`
`app/api/benchmark.py`	`/benchmark/runs`, xem lại và xuất CSV
`app/core/text.py`	Chuẩn hóa văn bản
`app/core/levenshtein.py`	DP chuẩn, DP có chặn, bảng DP và truy vết đường đi
`app/core/dictionary.py`	Nạp từ điển, khử trùng lặp, chỉ mục theo độ dài
`app/core/index.py`	Chỉ mục trigram sinh ứng viên
`app/core/suggester.py`	Xếp hạng top-k, bộ đệm LRU
`app/core/benchmark.py`	Đo hiệu năng, kho lưu kết quả, xuất CSV
`app/static/index.html`	Trang demo thử tay
`tests/`	117 test tự động
`benchmarks/benchmark.py`	Đo hiệu năng và kiểm tra chéo kết quả

---

5. Thuật toán và các tối ưu

5.1 Nền tảng: quy hoạch động Levenshtein

Khoảng cách Levenshtein giữa chuỗi `a` (truy vấn) và `b` (từ khóa) là số phép chèn, xóa hoặc thay thế ký tự ít nhất để biến chuỗi này thành chuỗi kia. Công thức truy hồi:

```
D[0][j] = j
D[i][0] = i
D[i][j] = min(
    D[i-1][j]   + 1,                    # xóa
    D[i][j-1]   + 1,                    # chèn
    D[i-1][j-1] + (a[i-1] != b[j-1])    # thay thế hoặc giữ nguyên
)
```

Ví dụ với `kitten` → `sitting` (giá trị 3 ở góc dưới phải):

	""	s	i	t	t	i	n	g
""	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

Chỉ cần giữ hai hàng của bảng, nên bộ nhớ là `O(min(m, n))`, thời gian `O(m·n)`. Đây chính là công thức trong `algorithm.py`; backend giữ nguyên để kết quả không đổi.

Xếp hạng: sắp theo `(distance, keyword)` tăng dần. Nhờ tiêu chí phụ là từ khóa, kết quả không phụ thuộc thứ tự duyệt của `set` hay nguồn dữ liệu — chạy lại luôn cho cùng một đáp án.

5.2 Chuẩn hóa tiếng Việt

Trước khi so sánh, cả từ điển và truy vấn đều được chuẩn hóa theo đúng quy tắc của `DictionaryProcessor`:

1. Chuẩn hóa Unicode về **NFC** (dạng dựng sẵn), tránh trường hợp `"ệ"` gõ theo hai kiểu tổ hợp khác nhau bị coi là hai ký tự khác nhau.
2. Chuyển thành chữ thường.
3. Bỏ ký tự không phải chữ, số, gạch dưới hoặc khoảng trắng.
4. Gộp khoảng trắng liên tiếp và cắt hai đầu.

Dấu tiếng Việt được giữ nguyên, nên `"điện thoại samsng"` gần `"điện thoại samsung"` (distance 1), còn `"dien thoai samsng"` xa hơn nhiều (distance 4) — đúng như bản gốc.

5.3 Tối ưu 1 — Lọc ứng viên bằng trigram

Vấn đề. Với mỗi truy vấn, tính khoảng cách cho cả 23.850 từ khóa tốn khoảng 600 ms.

Ý tưởng. Mỗi chuỗi được đệm hai khoảng trắng ở hai đầu rồi cắt thành các trigram (bộ ba ký tự liên tiếp). Ví dụ `"tai nghe"` (đã đệm) cho các trigram `" t"`, `" ta"`, `"tai"`, `"ai "`, `"i

n", …

Bổ đề. Một phép chèn, xóa hoặc thay thế ký tự chỉ có thể phá hủy tối đa 3 trigram của chuỗi gốc. Do đó nếu `distance(q, k) ≤ d` thì:

```
|G(q) ∩ G(k)| ≥ |G(q)| − 3d
```

trong đó `G(·)` là tập trigram phân biệt.

Chứng minh. Mỗi phép biến đổi chỉ ảnh hưởng tới các trigram chứa vị trí bị sửa: tối đa 3 trigram. Sau `d` phép, tối đa `3d` trigram của `q` bị phá hủy, phần còn lại buộc phải xuất hiện trong `k`.

Hệ quả. Điều kiện trên là điều kiện **cần**, nên dùng nó để loại ứng viên không bao giờ bỏ sót kết quả đúng. Với `"điện thoại samsng"` (19 trigram phân biệt) và `d = 1`, chỉ những từ khóa chia sẻ ít nhất 16 trigram mới được xét — thực tế còn đúng 1 ứng viên thay vì 23.850.

Nới ngưỡng dần. Chưa biết trước khoảng cách của kết quả thứ 5, nên backend thử `d = 1, 2, 3, ...`. Khi đã gom đủ 5 kết quả với ngưỡng `d`, mọi từ khóa có khoảng cách ≤ `d` đều nằm trong tập ứng viên; top-5 vì thế là **chính xác**, không phải gần đúng.

5.4 Tối ưu 2 — Quy hoạch động có chặn (dải chéo)

Khi chỉ cần biết khoảng cách có vượt ngưỡng `d` hay không, không cần tính cả bảng. Nếu khoảng cách cuối cùng ≤ `d` thì trên mỗi hàng `i`, chỉ các cột `j` thỏa `|i − j| ≤ d` mới có thể mang giá trị ≤ `d`.

Backend chỉ tính dải chéo rộng `2d + 1`, bỏ hẳn phần ngoài, và dừng ngay khi toàn bộ một hàng đều lớn hơn `d` (không còn đường đi rẻ hơn). Hàm trả về `d + 1` khi khoảng cách thật vượt ngưỡng — đủ để so sánh mà không cần con số chính xác.

Chi phí giảm từ `O(m·n)` xuống `O(m·d)`. Với `d = 1–5` và độ dài từ khóa 20–30 ký tự, mỗi phép tính chỉ còn vài chục ô thay vì hàng trăm.

5.5 Tối ưu 3 — Quét dự phòng theo độ dài

Với truy vấn quá ngắn hoặc quá xa từ vựng, `|G(q)| − 3d ≤ 0` nên bộ lọc trigram hết tác dụng. Lúc đó backend quét từ khóa theo nhóm độ dài lệch dần khỏi độ dài truy vấn.

Cơ sở: `distance(a, b) ≥ |len(a) − len(b)|`. Khi đã đủ 5 kết quả với khoảng cách tệ nhất `d`, mọi nhóm lệch quá `d` bị bỏ qua — vẫn đúng, chỉ chậm hơn. Đường dự phòng này cũng là lý do truy vấn ngắn như `"nồ"` trả về trong 0,3 ms.

5.6 Bộ đệm LRU

Kết quả được đệm theo khóa `(truy vấn chuẩn hóa, limit, max_distance)`, tối đa 1024 mục, có khóa luồng. Truy vấn lặp lại (người dùng gõ lại, demo trước lớp) trả về trong khoảng 0,02 ms. Số liệu hit/miss hiển thị ở `/api/v1/stats`.

5.7 Độ phức tạp tổng hợp

Trường hợp	Thời gian
Một cặp chuỗi, DP đầy đủ	`O(m·n)`
Một cặp chuỗi, DP có chặn	`O(m·d)`
Truy vấn khớp gần (d nhỏ)	`O((m + số ứng viên)·d)`
Truy vấn xa từ vựng	`O(N·m·d)` — quét toàn bộ
Cache hit	`O(1)`

Trong đó `N = 23.850` là số từ khóa, `m`, `n` là độ dài hai chuỗi, `d` là ngưỡng khoảng cách hiện tại.

---

6. Kiểm thử

6.1 Chiến lược

Bốn tầng, tổng cộng **69 test**:

Tầng	Tệp	Số test	Nội dung
Đơn vị	`test_text.py`	6	NFC/NFD, dấu câu, khoảng trắng, dữ liệu không hợp lệ
Đơn vị	`test_levenshtein.py`	7	Ca kinh điển, đối xứng, DP có chặn khớp DP đầy đủ trên 500 cặp ngẫu nhiên
Đơn vị	`test_levenshtein_trace.py`	7	Bảng DP và đường đi tối ưu trên 200 cặp ngẫu nhiên (dựng lại được chuỗi đích)
Đơn vị	`test_dictionary.py`	5	Nạp, khử trùng lặp, chỉ mục độ dài, tệp thiếu/rỗng
Đơn vị	`test_config.py`	9	Biến môi trường, đường dẫn tương đối, cấu hình sai
Đơn vị	`test_suggester.py`	13	So với brute-force, tie-break, `limit`, `max_distance`, cache, eviction
Đơn vị	`test_benchmark_core.py`	8	Quét toàn bộ, đo nguội, kho lưu run, CSV
Tích hợp	`test_api.py`	24	Endpoint, validation, CORS, header, trang demo, lỗi 404/405/500
Tích hợp	`test_contract_api.py`	33	Toàn bộ 10 endpoint theo `UI/src/API_List.md` + đối chiếu với `/api/v1`
Đối chiếu	`test_parity_original.py`	5	So trực tiếp với `algorithm.py`: khoảng cách, chuẩn hóa, top-5
Tổng		**117**	

Ngoài ra có kiểm tra thủ công bằng curl, thử 30 request đồng thời và đo hiệu năng bằng script riêng.

6.2 Kết quả

```
117 passed in 3.73s
Độ phủ: 100% (866 câu lệnh)
```

Chi tiết độ phủ:

Mô-đun	Độ phủ
`app/main.py`, `app/config.py`, `app/schemas.py`	100%
`app/api/*` (7 tệp)	100%
`app/core/*` (6 tệp)	100%
Toàn bộ `app/`	**100% (866/866 câu lệnh)**

6.3 Kiểm chứng không sai lệch so với bản gốc

Đây là phần quan trọng nhất: phần tối ưu chỉ được phép nhanh hơn, **không được đổi kết quả**.

Phép kiểm chứng	Số lượng	Kết quả
Khoảng cách so với `algorithm.py` trên cặp ngẫu nhiên	300 cặp	khớp 100%
Chuẩn hóa so với `DictionaryProcessor`	6 mẫu (NFC, NFD, dấu câu, chữ hoa)	khớp 100%
Top-5 so với bản gốc trên tập con 1.500 từ khóa	15 truy vấn gõ sai	khớp 100%
Top-5 so với bản gốc trên **toàn bộ** từ điển	2 truy vấn mẫu README	khớp 100%
Top-5 so với brute-force trên toàn bộ từ điển	12 truy vấn khó, kể cả truy vấn vô nghĩa	khớp 100%

Các truy vấn khó đã thử gồm: truy vấn không có trong từ điển (`nồi cơm điện tử`, `giày thể thao nam`), truy vấn rất ngắn (`nồ`, `a`), chuỗi chỉ có dấu câu, chữ hoa, và các trường hợp `max_distance` bằng 0, 1, 2, 3.

6.4 Kiểm thử thủ công

- Gọi đủ 4 endpoint cũ và 10 endpoint mới bằng curl trên từ điển 23.850 từ khóa, kiểm tra mã trạng thái và nội dung JSON.
- 30 request đồng thời: 0 lỗi; sau request đầu tiên, gần như toàn bộ các request còn lại được trả từ cache.
- Chạy `/benchmark/runs` với `include_baseline` trên từ điển thật: 2 truy vấn mẫu cho 6,9 ms so với 493 ms khi quét toàn bộ (nhanh hơn ~72 lần), top-k khớp bản gốc.
- Thử lỗi: 422 (dữ liệu sai), 404 (sai đường dẫn), 405 (sai phương thức), 500 (mô phỏng lỗi nội bộ) — tất cả trả JSON đúng cấu trúc, không lộ stack trace.

---

7. Kết quả đo hiệu năng

7.1 Môi trường đo

- Linux, Python 3.14.4, FastAPI 0.162, Uvicorn.
- Từ điển 23.850 từ khóa, `limit = 5`, không dùng cache (xóa trước mỗi lần đo).
- Bản gốc: `algorithm.py` chạy trên cùng từ điển, cùng máy.

7.2 Khởi động

Bước	Thời gian
Nạp và chuẩn hóa 23.850 từ khóa	38,4 ms
Dựng chỉ mục trigram + sắp xếp theo độ dài	65,0 ms
Tổng	**~103 ms**

7.3 Thời gian phản hồi

Truy vấn	Backend	Bản gốc	Nhanh hơn	Ứng viên đã tính
`điện thoại samsng`	11,7 ms	681 ms	×58	186
`tai nghe blutooth`	25,0 ms	654 ms	×26	809
`áo thun akko`	2,6 ms	500 ms	×196	84
`laptop dell i5`	3,0 ms	551 ms	×182	179
`bàn phím cơ`	3,2 ms	485 ms	×152	53

Với 20 truy vấn (5 mẫu trên + 15 truy vấn gõ sai ngẫu nhiên 1–3 ký tự):

Chỉ số	Giá trị
Trung bình	23,2 ms
p50	24,6 ms
p95	49,2 ms
Nhỏ nhất	2,6 ms
Lớn nhất (trong nhóm này)	53,8 ms
Cache hit	0,016 ms
So với bản gốc (trung bình 620 ms)	nhanh hơn ×26,7

7.4 Trường hợp xấu

Truy vấn **không có từ khóa nào gần** (ví dụ `nồi cơm điện tử`: bốn kết quả đầu cách 10 phép, kết quả thứ năm cách 11) khiến bộ lọc trigram hết tác dụng; backend phải quét đầy đủ 19.936 từ khóa và mất khoảng **705 ms** — xấp xỉ bản gốc. Kết quả vẫn đúng.

Đây là giới hạn của việc tính khoảng cách tuyệt đối trên toàn từ điển bằng Python thuần, không phải lỗi của bộ lọc. Với truy vấn thông thường (kể cả gõ sai), bộ lọc luôn phát huy tác dụng.

7.5 Cách tái lập

```
cd backend
python benchmarks/benchmark.py --queries 20 --naive 6
```

Script tự đối chiếu top-5 với bản gốc và báo lỗi nếu lệch; kết quả JSON đầy đủ nằm ở `report/benchmark-results.json`.

---

8. Hạn chế và hướng phát triển

Hạn chế	Hướng khắc phục
Truy vấn xa từ vựng mất ~0,7 s	Dùng chỉ mục lớn hơn (q-gram, BK-tree) hoặc thư viện C như `rapidfuzz`; đánh đổi bằng việc không còn tự cài thuật toán
Từ điển chỉ đọc, nạp lúc khởi động	Thêm endpoint nạp lại có xác thực, hoặc theo dõi thay đổi tệp
Chưa có xác thực, giới hạn tần suất	Thêm API key và rate limit khi triển khai công khai
Chưa tối ưu riêng cho gợi ý theo từng ký tự	Thêm chế độ prefix/autocomplete với chỉ mục riêng
Chạy một tiến trình	Có thể chạy nhiều worker Uvicorn; từ điển chỉ đọc nên an toàn

---

9. Kết luận

Backend đã hoàn thành mục tiêu: biến thuật toán Levenshtein của nhóm thành một dịch vụ HTTP đúng, nhanh và có kiểm chứng.

- **Đúng:** kết quả trùng khớp 100% với `algorithm.py`, được kiểm tra tự động trên

toàn bộ 23.850 từ khóa cùng nhiều truy vấn khó.

- **Nhanh:** từ ~600 ms xuống 2,5–25 ms cho truy vấn thông thường (nhanh hơn ~26 lần),

nhờ lọc trigram có chứng minh không bỏ sót, DP dải chéo và cache.

- **Chắc:** 117 test, độ phủ 100%, xử lý lỗi thống nhất, có log và số liệu vận hành.
- **Dễ dùng:** chạy bằng hai lệnh, có Swagger UI, trang demo và tài liệu đầy đủ.

Phần tối ưu không thay đổi bản chất thuật toán của môn học: vẫn là quy hoạch động Levenshtein, chỉ thêm các lớp cắt tỉa có chứng minh để dùng được trong thực tế.

---

10. Phụ lục

A. Hướng dẫn chạy

```
cd backend
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt

uvicorn app.main:app --reload    # http://127.0.0.1:8000
python -m pytest                 # 117 test
python benchmarks/benchmark.py   # đo hiệu năng
```

B. Danh sách endpoint

Phương thức	Đường dẫn	Mô tả
POST	`/api/v1/suggest`	Gợi ý từ khóa, body `{query, limit?, max_distance?}`
GET	`/api/v1/suggest?q=...&limit=5`	Gợi ý từ khóa qua query string
GET	`/api/v1/health`	Trạng thái dịch vụ

Phương thức	Đường dẫn	Mô tả
GET	`/api/v1/stats`	Thống kê từ điển, cache, thuật toán
GET	`/docs`, `/redoc`	Tài liệu tương tác

Nhóm đường dẫn gọn theo hợp đồng frontend (`UI/src/API_List.md`):

Phương thức	Đường dẫn	Mô tả	
GET	`/health`	Trạng thái dịch vụ	
GET	`/search/suggest?q=...&limit=5&max_distance=`	Gợi ý mờ khi gõ sai	
GET	`/search?q=...&limit=20&mode=contains\`	prefix`	Tìm từ khóa chứa/bắt đầu bằng chuỗi
GET	`/dictionary?q=&length=&page=1&page_size=50&sort=alphabet\`	length`	Duyệt từ điển
GET	`/dictionary/stats`	Thống kê từ điển, thuật toán, cache	
GET	`/algorithm/distance?source=...&target=...&normalize=true`	Khoảng cách Levenshtein	
POST	`/algorithm/visualize`	Bảng DP đầy đủ + đường đi tối ưu (tối đa 60 ký tự/chuỗi)	
POST	`/benchmark/runs`	Chạy benchmark, trả `run_id`	
GET	`/benchmark/runs/{run_id}`	Xem lại kết quả	
GET	`/benchmark/runs/{run_id}/export`	Tải CSV (UTF-8 BOM)	

Mã lỗi: 422 (dữ liệu không hợp lệ), 404 (không tìm thấy), 405 (sai phương thức), 500 (lỗi máy chủ). Mọi lỗi trả về `{"error", "message", "details"?}`.

C. Biến môi trường

Biến	Mặc định	Ý nghĩa
`DICTIONARY_PATH`	`../dictionary.txt`	Đường dẫn từ điển
`SUGGEST_CACHE_SIZE`	`1024`	Số mục cache, 0 = tắt
`CORS_ORIGINS`	`*`	Origin được phép, cách nhau bởi dấu phẩy
`LOG_LEVEL`	`INFO`	Mức log

D. Ví dụ gọi API

```
curl -X POST http://127.0.0.1:8000/api/v1/suggest \
  -H 'Content-Type: application/json' \
  -d '{"query": "tai nghe blutooth", "limit": 3}'
```

```
{
  "query": "tai nghe blutooth",
  "normalized_query": "tai nghe blutooth",
  "suggestions": [
    { "keyword": "tai nghe bluetooth", "distance": 1 },
    { "keyword": "tai nghe hp bluetooth", "distance": 4 },
    { "keyword": "tai nghe lg bluetooth", "distance": 4 }
  ],
  "count": 3,
  "limit": 3,
  "max_distance": null,
  "took_ms": 11.3,
  "cached": false,
  "evaluated_candidates": 534
}
```


Ví dụ gọi nhóm endpoint mới:

```
curl "http://127.0.0.1:8000/dictionary?q=samsung&page_size=2"
curl "http://127.0.0.1:8000/search?q=tai%20nghe&limit=2"
curl -X POST http://127.0.0.1:8000/algorithm/visualize \
  -H 'Content-Type: application/json' -d '{"source": "kitten", "target": "sitting"}'
curl -X POST http://127.0.0.1:8000/benchmark/runs \
  -H 'Content-Type: application/json' \
  -d '{"queries": ["tai nghe blutooth", "áo thun akko"], "include_baseline": true}'
```

Ví dụ kết quả `/benchmark/runs` trên từ điển 23.850 từ khóa:


```
{
  "run_id": "224df5c36b28",
  "status": "completed",
  "duration_ms": 998.9,
  "summary": {
    "query_count": 2,
    "mean_ms": 6.852,
    "p95_ms": 11.114,
    "baseline_mean_ms": 492.548,
    "speedup_vs_baseline": 71.889,
    "baseline_mismatches": 0
  }
}
```


E. Tài liệu tham khảo

1. Levenshtein, V. I. (1966). *Binary codes capable of correcting deletions, insertions, and reversals*. Soviet Physics Doklady.

2. Ukkonen, E. (1985). *Algorithms for approximate string matching*. Information and Control.

3. FastAPI Documentation — <https://fastapi.tiangolo.com>

4. Nguồn mã nguồn môn học: <https://github.com/ngphuc-ict/levenshtein-keyword-suggestion>